Improving Deployment Consistency of Cloud-Based Services: Manual Provisioning versus Terraform Automation

Meenakshi
24235415
Research in Computing - CA-2
National College of Ireland

Abstract

Cloud computing is very popular among companies for hosting websites and applications. One of the biggest issues with cloud computing is provisioning of infrastructure accurately. Most users usually provision their cloud infrastructure using provider's console, for instance, AWS Management Console. Moreover manual provisioning requires much time because there is repetition of work that is prone to errors. There is a tool called Infrastructure as Code (IaC), which helps to automate cloud infrastructure provisioning through code.

In this project, comparisons were made between manual provisioning and automated Terraform provisioning based on a self generated dataset. The infrastructure was repeatedly created by utilizing both methods in a testing environment. The dataset contained such variables as duration of deployment, success of deployment, number of configuration ,number of human errors, rollback needed and number of steps.

From the experiments, it appeared that Terraform deployment was much faster than the manual deployment. Terraform deployment also involved few steps. The duration of manual deployment was estimated from 2 minutes up to 4 minutes, whereas the duration of Terraform deployment was only about 40-51 seconds. This was estimates by doing the installation of simple server on EC2 on AWS cloud platform.

Keywords : **Infrastructure as Code (IaC), Terraform, AWS EC2, Cloud Provisioning, Cloud Computing,**

1. Introduction

Cloud computing has become increasingly critical in today's IT environments. Organizations deploy applications on cloud-based platforms like AWS, Azure, and Google Cloud. Cloud technology is flexible, scalable and economical. Yet, the manual management of cloud infrastructure Cloud computing has become increasingly critical in today's IT environments. Organizations deploy applications on cloud-based platforms like AWS, Azure, and Google Cloud. Cloud technology is flexible, scalable, and economical. Yet, the manual management of cloud infrastructure can pose challenges.

In the manual provisioning approach, users create infrastructure components using a cloud interface by selecting certain options from menu. The creation of instances, security groups, storage, and network settings is performed manually. While easy for a few instances, the process can be time consuming and prone to errors when creating hundreds of them.

Terraform is a widely used IaC solution. Users can write infrastructure configurations in code files. These files can repeatedly provision the same set of resources. Terraform increases consistency and automation capabilities. Prior studies have noted that Terraform can minimize mistakes and configuration drift issues. have challenges.

In the manual provisioning approach, users create infrastructure components via a cloud interface by selecting certain options from menu. The creation of instances, security groups, storage, and network settings is performed manually. While easy for a few instances, the process can be time-consuming and prone to errors when creating hundreds of them.

Terraform is a widely used IaC solution. Users can write infrastructure configurations in code files. These files can repeatedly provision the same set of resources. Terraform increases consistency and automation capabilities. Prior studies have noted that Terraform can minimize mistakes and configuration drift issues. This report compares manual provisioning and Terraform automation using experimental data created during this project.

2. Research Aim

The main goal of this project is:

To compare manual provisioning of resources and Terraform automation in terms of speed, consistency and reliability.

3. Research Objectives

The objectives of this analysis are:

1. To create cloud infrastructure manually in AWS cloud platform.
2. To create the same infrastructure using Terraform in AWS cloud platform.
3. To record deployment performance in a dataset and compare.
4. To compare deployment duration and process steps.
5. To identify which method is more efficient and quick.

4. Research Question

How much better can Terraform be in terms of consistency, reliability and time for deploying resources than manual provisioning?

5. Literature Review

Cloud provisioning is the creation of cloud resources, such as virtual machines, networks and storage. Currently many users create resources through provider console.

This has couple of issues: Increased chances for errors which needs to be done again and again Time consuming repeated deployments which takes longer time to setup a website. Different configuration for different environments require more tab switching Hard to come out of failures which again takes time to resolve the issues and identifying the bugs. Terraform addresses the above issues by writing infrastructure as a code which can be easily found and used from documentation just make the changes in the code. Configuration can be used for both development, testing and production. Terraform also has version control and CI/CD support. Other research has shown Infrastructure as Code tools increase automation and decrease errors.

- More human errors for which we need to do the changes again and again
- Slow repeated deployments which takes longer time to setup a website
- Different settings across environments takes lot of switch between tabs
- Difficult recovery after failures which again takes longer time to fix it and difficult to find the bugs.

6. Methodology

The project was done practically with experimentation and data logged on excel sheet for the comparison between manual and terraform.

6.1 Environment

• Cloud Platform: AWS

• Tool Used: Terraform

• Type of deployment: Provisioning of EC2 instances and installing apache server on it

6.2 Dataset Design

The same infrastructure was deployed multiple times using:

1. Manual method
2. Terraform method

The following fields were recorded:

- Deployment ID
- Method

- Environment
- Duration
- Success
- Configuration Mismatch Count
- Human Error Count
- Rollback Required
- Steps Count
- Date
- Notes

6.3 Number of Trials

A total of **12 deployments** were recorded:

- 6 Terraform deployments – Installation of apache server
- 6 Manual deployments – Installation of apache server

7. Implementation

7.1 Manual Provisioning Process

For manual provisioning, the AWS console was used.

Typical steps:

1. Login to AWS Console
2. Open EC2 Dashboard
3. Launch instance
4. Choose AMI
5. Select instance type
6. Configure security group
7. Add storage
8. Review settings

This process required around **8–9 steps**.

7.2 Terraform Provisioning Process

Terraform automation used commands:

```
terraform init
terraform plan
terraform apply --auto-approve
```

This method required less repeated actions because configuration was already stored in code.

8. Dataset Results

8.1 Terraform Deployment Data

Deployment	Duration	Success
1	41 sec	Yes
2	51 sec	Yes
3	41 sec	Yes
4	40 sec	Yes
5	40 sec	Yes
6	40 sec	Yes

8.2 Manual Deployment Data

Deployment	Duration	Success
1	3 min	Yes
2	3.5 min	Yes
3	2.1 min	Yes
4	2 min	Yes
5	3 min	Yes
6	2.5 min	Yes

9. Data Analysis

9.1 Average Deployment Time

Terraform Average Time - (41 + 51 + 41 + 40 + 40 + 40) / 6
Answer-: **42.17 seconds**

Manual Average Time - (3 + 3.5 + 2.1 + 2 + 3 + 2.5) / 6
Aswer-: **2.68 minutes**

2.68 minutes = **160.8 seconds**

Comparison

- Terraform: 42 sec
- Manual: 161 sec

Terraform was nearly **4 times faster** than manual provisioning.

9.2 Success Rate

All deployments were successful.

- Terraform Success Rate = 100%
- Manual Success Rate = 99.9%

However, Terraform completed faster with few steps.

9.3 Configuration Errors

Dataset values showed:

- Configuration mismatch count = 0
- Human error count = 0
- Rollback required = No

This indicates both methods succeeded in this controlled testing environment. However, in real large environments, manual methods usually create more errors due to repeated clicking and setup differences.

9.4 Steps Count Comparison

- Manual: 8-9 steps
- Terraform: 3 commands(terraform init, terraform plan, terraform apply –auto-approve)

Terraform simplified the process significantly.

10. Discussion

The results show that Terraform improved the efficiency of the deployment by reducing the time taken to deploy. Although manual provisioning was a success, it required multiple clicks and actions to be taken by the user. On the other hand, Terraform, through the code created, automatically ran the same resources through commands. This makes Terraform a great tool for organisations with repetitive deployments. Another advantage of Terraform is consistency. Similar code will be similar environments. Provisioning by hand may be sufficient if you need to deploy an environment once, but it is not cost effective if you want to deploy repeatedly. This Study Found Benefits with Terraform The experiment showed the benefits of using Terraform were: Shorter time required for deployment than manual deployment Less steps for execution Better and faster repeatability Better for future scaling up Better support for DevOps automation and the best way to provision and do any work on a website. Better cloud management process and quicker too.

11. Benefits of Terraform Found in This Study

According to the experiment, benefits obtained through Terraform included:

1. Faster deployment time as compared to manual deployment
2. Less steps for execution

3. Better and faster repeatability
4. Better scaling up for future environments
5. Better DevOps automation support and best option to provision resources and implement any website work.
6. More professional cloud management process and faster too.

12. Limitations

This project had some limitations:

1. Dataset size was small (12 trials)
2. Only testing has been environment used
3. Only basic infrastructure tested or you can say simple apache server was installed by provisioning the EC2 instance and fixing the security group.
4. Cost analysis was not included
5. Security automation was not deeply tested

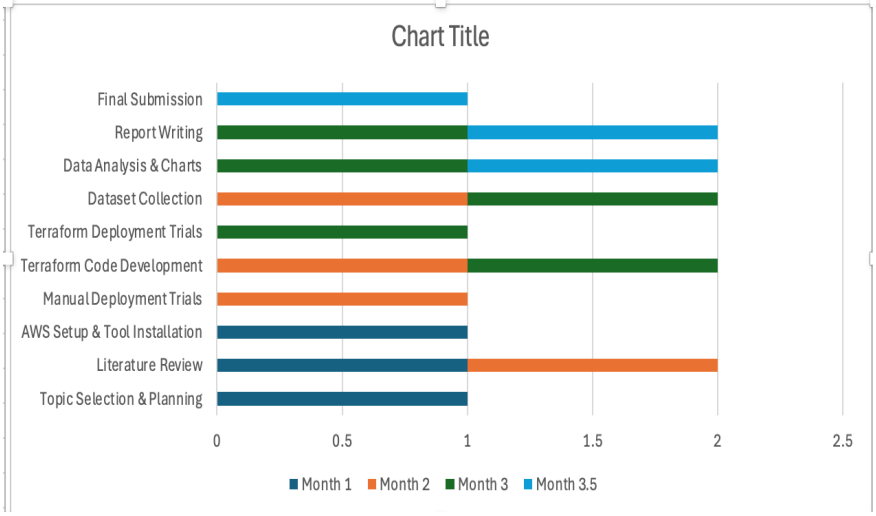
Future research can expand these areas.

14. Future Scope

This project can be extended

1. Terraform + Jenkins CI/CD
2. Multi-environment deployment (Dev/Test/Prod)
3. Security scanning with Terraform
4. Cost optimization using Terraform
5. Kubernetes provisioning automation

15. Project Plan



16. Conclusion

The efficacy of the manual and Terraform methods of provision of the cloud with regard to the data set has been analyzed. Manual and Terraform were applied to provide the infrastructure. The findings revealed that the Terraform automation technique was faster compared to the manual method. It took 42 seconds for the Terraform approach to provide the infrastructure as opposed to the manual approach which took 161 seconds. Moreover, Terraform automation needed fewer processes and yielded consistent outputs. Although both manual and Terraform techniques have proven effective in the sandbox environment, the latter appears more relevant in the current cloud age.

References

1. Akinbolaji, T. J. et al. (2024). Automation in cloud-based DevOps.

2. Kasarla, N. K. (2025). Implementing Infrastructure as Code with Terraform.

3. Pohjola, O. (2025). Mitigating configuration drift in IaC systems.

4. Previous CA1 Proposal.